

Week 7 & 8 Workplan: Dashboard & Data Product Build

Two-week plan for a CPF Group multi-subsidiary analytics dashboard prototype

For this task I'll build an analytics dashboard spanning three CPF Group subsidiaries, so the prototype demonstrates a genuine portfolio view instead of a single product's numbers in isolation.

Rukisha, the group's mobile wallet for credit access, merchant payments, fund transfers and goal based and pension linked savings, is the anchor because its data (loans, disbursements, repayments, defaults) gives me an actual relational model to design around. Alongside it I'm adding CPF Financial Services, whose mission is to provide diversified, innovative and sustainable financial solutions to our stakeholders in the region through pension funds administration, trust fund administration and agency services, and CPF Capital & Advisory, positioned around catalysing alternative investments and advancing capital markets, with a track record that includes the Laptrust IMARA I-REIT (the region's only pension backed REIT), the Talanta Stadium financing and the Linzi Sukuk KDF Housing project. Together the three give the dashboard lending, pensions and capital markets as distinct business lines to model side by side. The stack is Next.js on Vercel with a Postgres database (Supabase), Python and pandas to generate and load dummy data for all three, and the repo on GitHub feeding a CI/CD pipeline into Vercel.

Rather than starting from the UI, I'm running this the way the Software Development Life Cycle (SDLC) I documented in Week 6 lays out: plan and analyse the requirement first, design the system on paper, then build, test and deploy, in that order. Week 7 covers the first three of those phases; Week 8 covers the rest. The core design principles and clean coding practices from that same week aren't just theory here, so I've noted below exactly where each one shows up in the build.

Week 7 - Planning, Analysis and Design

This week is deliberately front-loaded with the non-coding phases of the SDLC, since jumping straight to code is the exact mistake that framework exists to prevent.

- **Planning:** A short scope note covering what each subsidiary's view needs to show, portfolio value, disbursement trend, repayment and default rate for Rukisha; member contributions and fund balance for CPF Financial Services' pension book; deal count and transaction value for CPF Capital & Advisory, who it's for (a CPF Group finance manager comparing business lines), and what's out of scope for a two-week build (no real authentication, no live data feed).
- **Analysis:** Turning each of those requirements into something concrete enough to design against. "Show default rate by loan product" becomes a defined calculation and data source for Rukisha, and the same discipline carries over to defining fund balance growth for pensions and deal value for capital markets, not just phrases on a wishlist.
- **Design:** A short Software Design Document, essentially the blueprint for the system before any code exists, covering the architecture (how the Next.js app talks to Postgres), an entity relationship diagram for the database, and how the components break down. A subsidiary field ties every table back to Rukisha, CPF Financial Services or CPF Capital & Advisory, so the three business lines sit on one schema instead of three disconnected ones. This is where separation of concerns and single responsibility get applied on paper: the data fetching layer, the business logic (default rate, fund balance and deal value calculations), and the UI components stay as distinct pieces, each with one clear job.
- **Environment and CI/CD:** Setting up the GitHub repository and connecting it to Vercel so every push to main triggers an automatic build and deploy, plus a GitHub Actions workflow that runs the linter and a build check on every push. This is the same automated tooling habit from the Week 6 clean coding file, catching problems before they reach the main branch instead of relying on me to remember to check.

Week 8 - Development, Testing and Deployment

With the design already settled, this week is where most of the actual coding happens, the goal is to build against the design from Week 7, not to keep redesigning as I go.

- **Development:** The data access layer and dashboard components get built against the Week 7 schema, with a subsidiary selector driving which business line's view is shown. Clean coding practices apply directly here, descriptive naming instead of shorthand variables, small functions that each do one thing, and no duplicated logic (the DRY principle): each subsidiary's key figure, default rate, fund balance growth, deal value, is calculated once and reused across every chart that needs it.
- **Testing:** Before calling it done, I'll manually test each subsidiary's view against edge cases, no data for a given filter, a loan with no repayments yet, a pension scheme with no contributions this period, and fix what breaks. For a two-week prototype, structured manual QA against the Week 7 requirements is the right scope rather than a full automated test suite.
- **Deployment:** The GitHub Actions pipeline deploys to Vercel automatically on merge to main; I confirm the hosted version matches what was tested locally across all three subsidiary views.
- **Documentation:** The one-page design and architecture brief the assignment calls for, explaining in plain terms which SDLC phases were followed and where in the actual code the design principles and clean coding practices show up, not just an assertion that they were applied.